Thread Reader

Nikhil Ghosh @nikhilghosh101
Jan 8 • 12 tweets

Tweet Share Download

Sharing our recent work on understanding the mechanisms underlying the empirical success of hyperparameter transfer using μP! (1/11)

with Denny Wu and @Alberto Bietti

Understanding the Mechanisms of Fast Hyperparameter Transfer

Nikhil Ghosh[1], Denny Wu[1,2], Alberto Bietti[1]
[1]Flatiron Institute, [2]New York University
{nghosh,dwu,abietti}@flatironinstitute.org

January 5, 2026

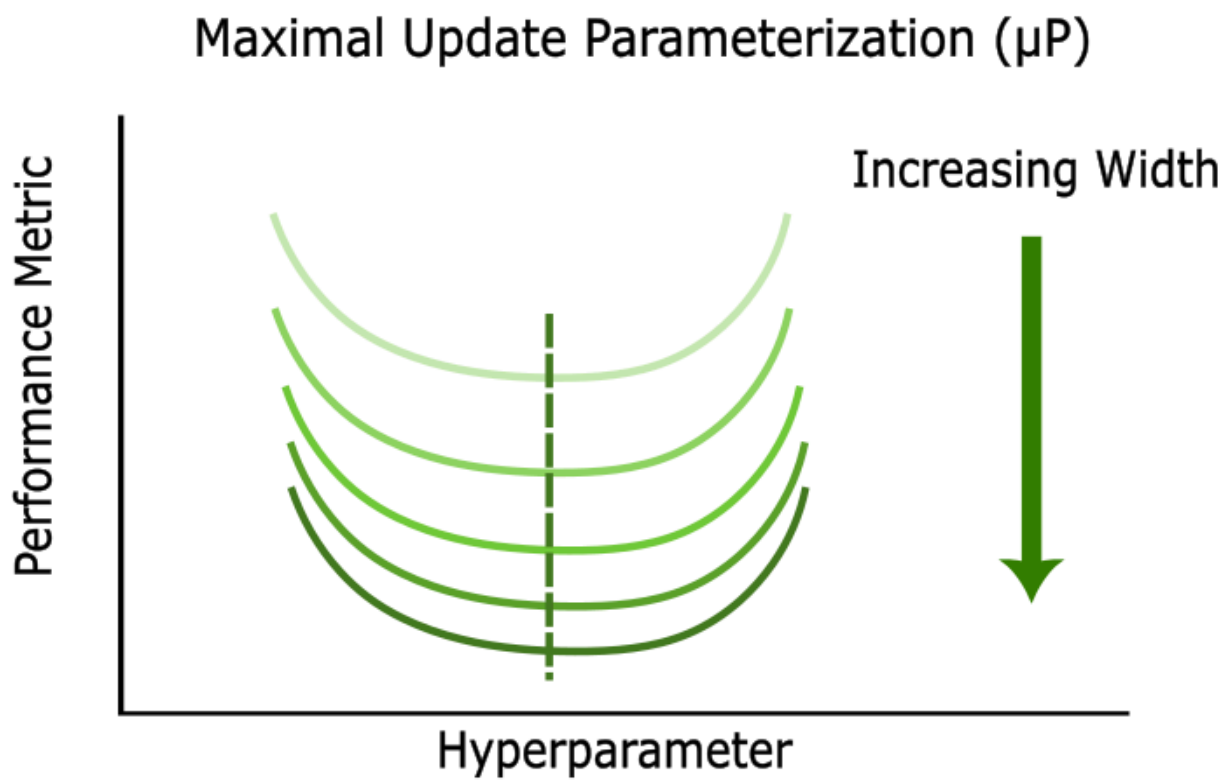
Abstract

The growing scale of deep learning models has rendered standard hyperparameter (HP) optimization prohibitively expensive. A promising solution is the use of scale-aware hyperparameters, which can enable direct *transfer* of optimal HPs from small-scale grid searches to large models with minimal performance loss. To understand the principles governing such transfer strategy, we develop a general conceptual framework for reasoning about HP transfer across scale, characterizing transfer as *fast* when the suboptimality it induces vanishes asymptotically faster than the finite-scale performance gap. We show formally that fast transfer is equivalent to *useful* transfer for compute-optimal grid search, meaning that transfer is asymptotically more compute-efficient than direct tuning. While empirical work has found that the Maximal Update Parameterization (μP) exhibits fast transfer when scaling model width, the mechanisms remain poorly understood. We show that this property depends critically on problem structure by presenting synthetic settings where transfer either offers provable computational advantage or fails to outperform direct tuning even under μP. To explain the fast transfer observed in practice, we conjecture that decomposing the optimization trajectory reveals two contributions to loss reduction: (1) a width-stable component that determines the optimal HPs and (2) a width-sensitive component that improves with width but weakly perturbs the HP optimum. We present empirical evidence for this hypothesis across various settings, including large language model pretraining.

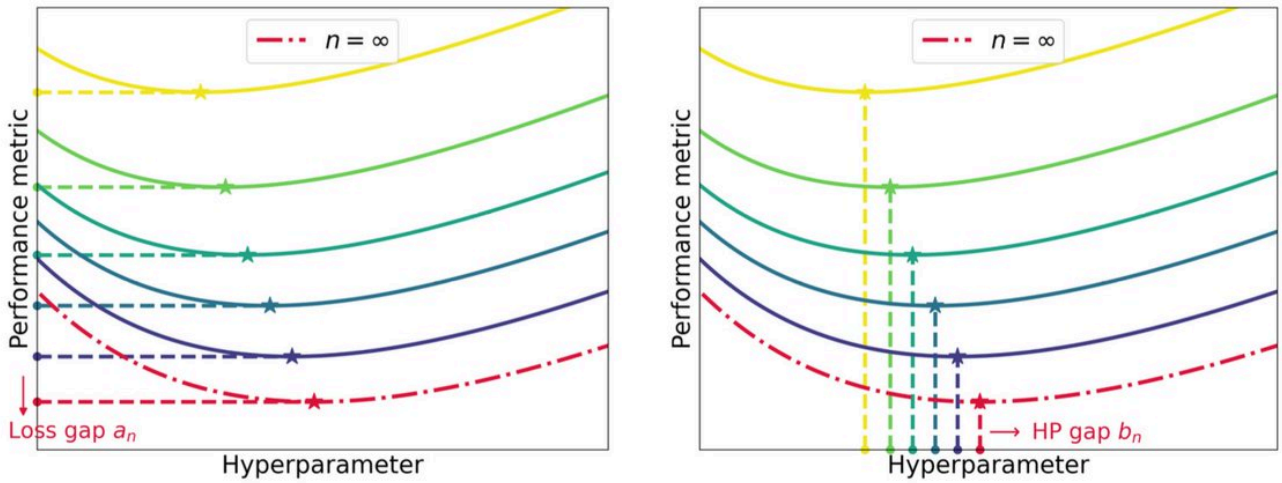
Hyperparameter (HP) tuning is brutally expensive at scale.

The Maximal Update Parameterization (μP) from offers a cheap solution: tune HPs on a small proxy model, then *transfer* them to a large-scale run.

But when does this actually work—and why? (2/11)



First, what does it mean for transfer to “work”? μP is designed so optimal HPs have a width-independent limit, but says nothing about rates—so small proxies can provide a poor guess. As width n grows, the HP gap b_n must shrink “fast enough” relative to the loss gap a_n. (3/11)



(a) Loss gap a_n. (b) Hyperparameter gap b_n.

Under mild assumptions, b_n = O(a_n^(1/2)).
We say that HP transfer is *fast* when b_n beats this a_n^(1/2) rate.

We prove this is exactly when transfer is *useful*: grid search with transfer is asymptotically more compute-efficient than direct tuning. (4/11)

Theorem 2 (Useful transfer). *Suppose $a_n \sim n^{-\alpha}$ and $b_n \sim n^{-\beta}$. We are given a compute budget of $\mathcal{F}$ flops, we tune h different HPs, and a single training run at width n costs n^r flops.*

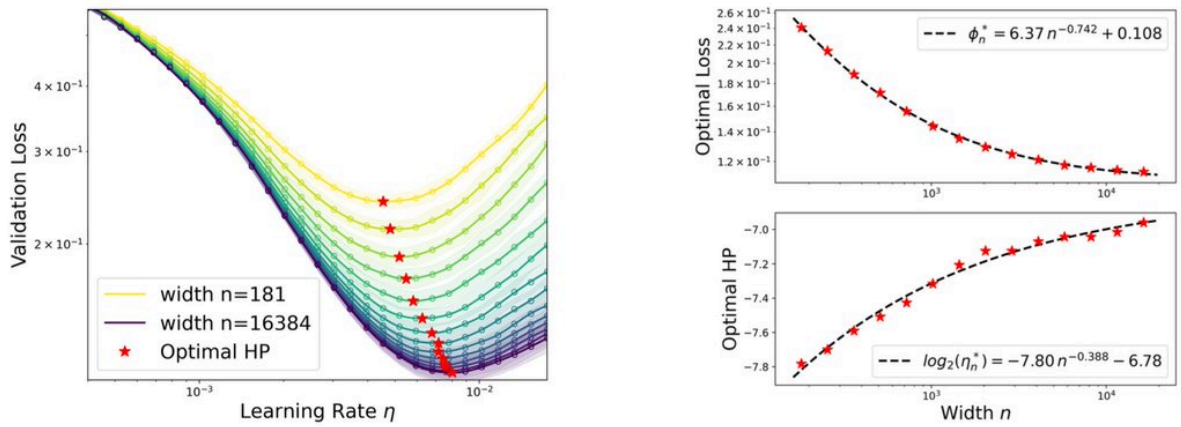
(a) ***Direct tuning.*** *If we directly conduct grid search on a width-n model, the compute-optimal performance scales as $\mathcal{F}^{\frac{-2\alpha}{h\alpha+2r}}$ and is obtained at width $n^\star \sim \mathcal{F}^{\frac{2}{h\alpha+2r}}$.*

(b) ***Transfer.*** *If we conduct a grid search on a width-n model and then transfer to a large width-M model, the compute-optimal performance scales as $\mathcal{F}^{\frac{-\alpha}{r}} + \mathcal{F}^{\frac{-2\beta}{h\beta+r}}$, obtained at widths $n^\star \sim \mathcal{F}^{\frac{1}{h\beta+r}}$ and $M^\star \sim \mathcal{F}^{1/r}$.*

Hence transfer is useful if and only if $\beta > \alpha/2$, i.e., $b_n = o(\sqrt{a_n})$.

It turns out that transfer is not fast by default: in a synthetic example, we find b_n ~ a_n^(1/2), matching the generic bound.

Thus, fast transfer in natural settings must rely on additional “structure.” (5/11)



(a) Validation loss vs. learning rate. (b) Convergence of optimal loss and learning rate.

Figure 5: Optimal learning rate (validation loss) for two-layer ReLU network to learn the ball indicator function.

What kind of structure? Intuitively, optimal HPs should depend mainly on network statistics that stabilize with width much faster than the loss itself.

We formalize this intuition via a low-rank decomposition of the optimization trajectory and connect it to fast transfer. (6/11)

Key idea: the stepwise linearized loss change <G, dW> decomposes into top-k and residual contributions. Summing over time and layers defines the top-k loss and residual loss.

Empirically, the linearization tracks actual loss change for EMA-smoothed trajectories. (7/11)

Matrix parameter $\boldsymbol{W} \in \mathbb{R}^{m\times n}$, gradient $\boldsymbol{G} \in \mathbb{R}^{m\times n}$, and update $\delta\boldsymbol{W} \in \mathbb{R}^{m\times n}$.

$$\boldsymbol{A} := \tfrac{1}{2}\left(\boldsymbol{G}^\top \delta\boldsymbol{W} + \delta\boldsymbol{W}^\top \boldsymbol{G}\right) \quad \text{(alignment matrix)}$$
$$|\lambda_1| \geq \cdots \geq |\lambda_n| \quad (\lambda_i \text{ eigenvalues of } \boldsymbol{A})$$

Stepwise linearized loss change for parameter $\boldsymbol{W}$:

$$\delta\phi(\boldsymbol{W}) := \langle \boldsymbol{G}, \delta\boldsymbol{W}\rangle = \underbrace{\sum_{i=1}^{k} \lambda_i}_{\delta\phi^k(\boldsymbol{W})} + \underbrace{\sum_{i=k+1}^{n} \lambda_i}_{\delta\phi^{-k}(\boldsymbol{W})}.$$

Summing over layers ℓ and time steps t:

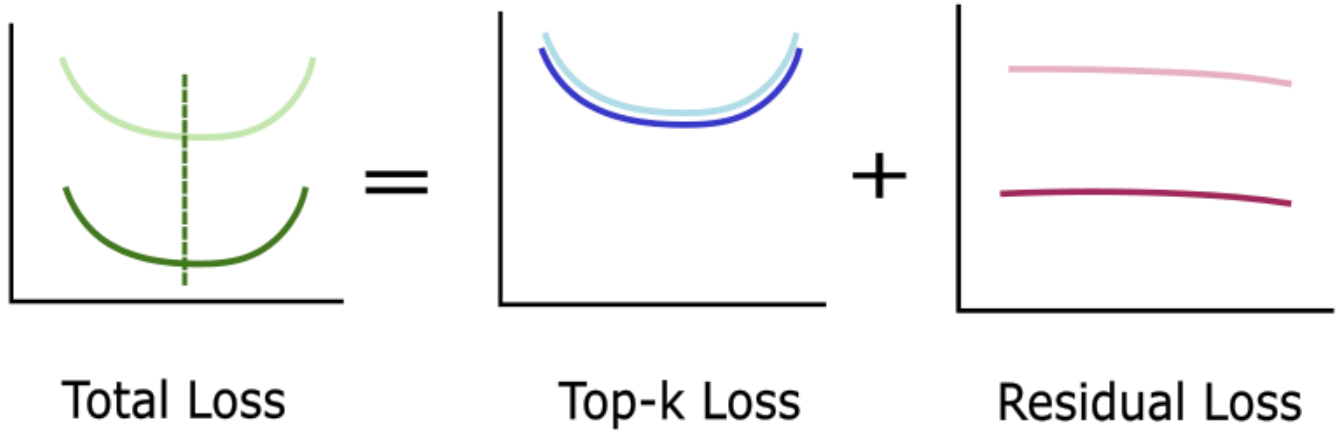
$$\underbrace{\mathcal{L}(\boldsymbol{w}_T) - \mathcal{L}(\boldsymbol{w}_0)}_{\text{total loss}} \approx \underbrace{\sum_{\ell=1}^{L}\sum_{t=0}^{T-1} \delta\phi^k\left(\boldsymbol{W}_t^{(\ell)}\right)}_{\text{top-}k\text{ loss}} + \underbrace{\sum_{\ell=1}^{L}\sum_{t=0}^{T-1} \delta\phi^{-k}\left(\boldsymbol{W}_t^{(\ell)}\right)}_{\text{residual loss}}.$$

Our hypothesis is that fast transfer occurs when for some k < n,

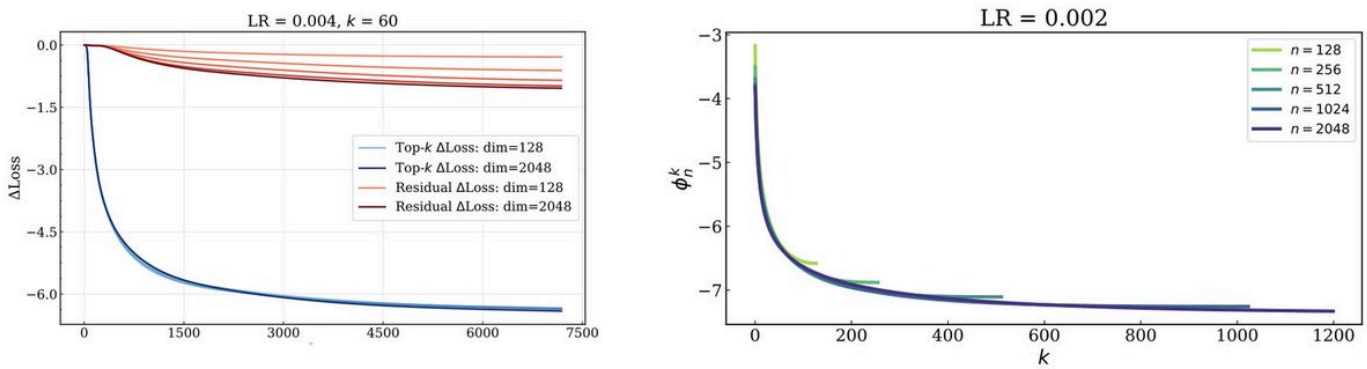
(1) the top-k loss converges much faster than the overall loss.

(2) the optimal HPs of the top-k loss "determine" the overall optimal HPs because the residual loss is relatively insensitive to HPs.

(8/11)



Empirically, the top-k loss captures most of the loss reduction and varies far less across widths than the residual. Width gains come mainly from the residual late in training. As k grows, small-width top-k losses overlap large-width ones, then “peel off” and plateau. (9/11)



Setup: 4-layer LLaMA • WikiText-103 • Adam • warmup–stable–decay (WSD) LR

If we pick k as the “peel-off” index from n=2048. Then small+large widths have nearly identical top-k loss and share the same optimal LR as the total loss; the residual is flat and doesn’t move the optimum. This matches the fast-transfer structure we conjectured. (10/11)

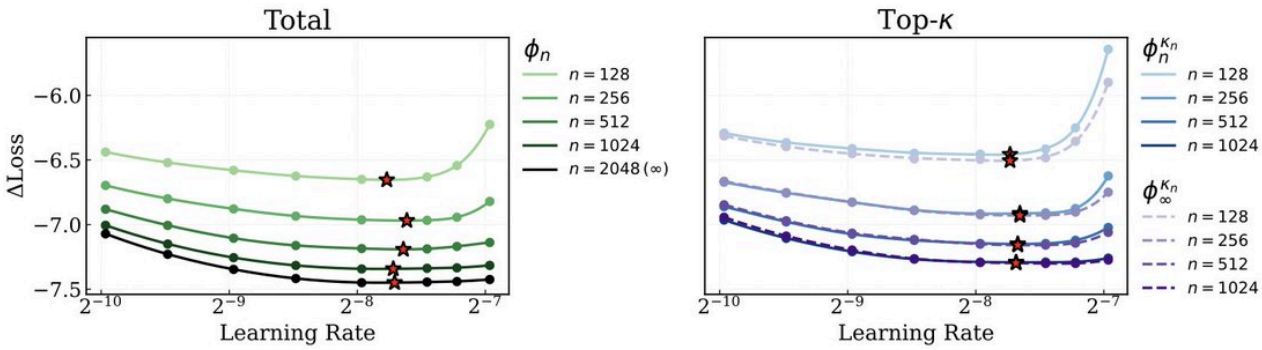


Figure 8: **Left:** Total loss curves ϕ_n across widths. **Right:** Top-κ_n loss curve pairs $\phi_n^{\kappa_n}$ (blue dashed) and $\phi_\infty^{\kappa_n}$ (purple dashed). The top-κ_n pairs nearly overlap, with minimizers close to those of the corresponding total losses.

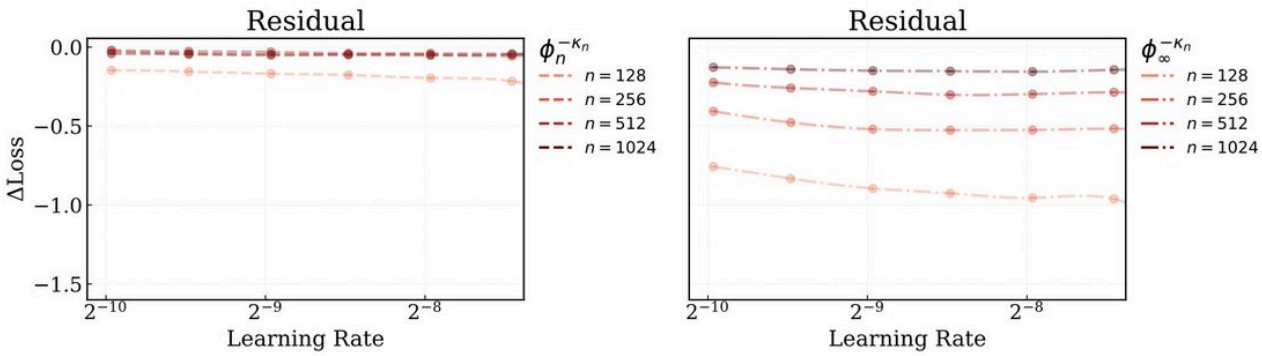
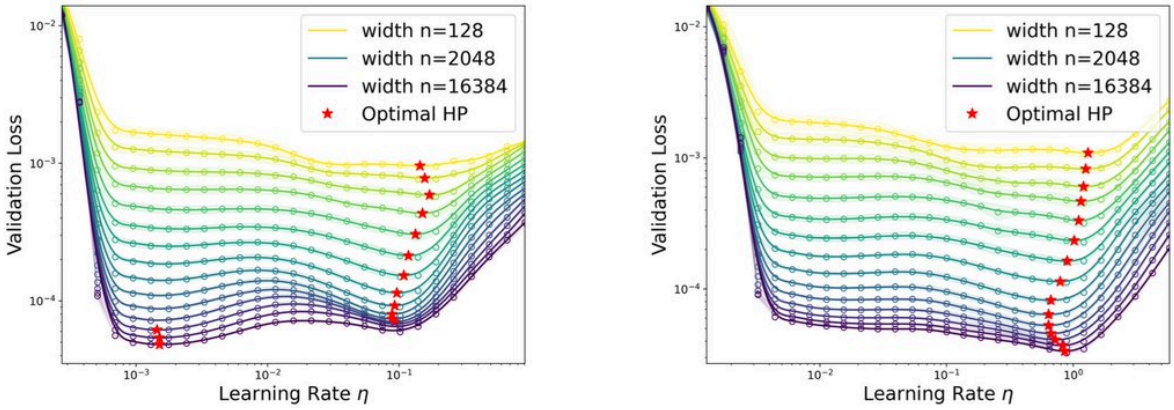


Figure 9: Residual losses are flat around the top-κ_n minimizers, indicating less sensitivity to the learning rate.

Please see our paper arxiv.org/abs/2512.22768 for much more:

1. an interesting μP failure case,
2. provably fast transfer for RF ridge regression,
3. transfer for Muon and Dion,
4. a data-centric viewpoint of the top-k loss and transfer.

(11/11)



(a) Identical all-layer learning rate (Figure 2).

(b) Different layer-wise learning rate.

Figure 17: Learning Gaussian k-index model with two-layer ReLU network under μP. The HP of interest is the Adam learning rate. **Left:** default μP implementation with the same learning rate for the first- and second-layer parameters. **Right:** layer-wise learning rate where the second-layer learning rate is divided by 50. Observe that the original μP yields a "bimodal" HP curve, whereas layer-wise HP leads to more stable transfer.

If you are interested in working on similar questions with us at the Flatiron Institute this summer, please consider applying for an internship and reaching out!

Nikhil Ghosh

@nikhilghosh101

Research Fellow at Flatiron Institute. prev: PhD at UC Berkeley Statistics

Follow on 𝕏

twitter-thread.com/t/2009278025959903525